

Komponistenklassifikation und Musikgenerierung mit maschinellem Lernen

Ferdinand Beck, Anna Bergner, Kristian Kelly, Severyn Khomych, Ines Landauer

24. Februar 2026

Zusammenfassung

In dieser Arbeit beschäftigen wir uns mit der Klassifikation von Komponisten anhand ihrer Werke sowie der Generierung von Musikstücken. Für Ersteres verwenden wir 'klassische' Machine Learning-Modelle wie Support Vector Machines und Random Forests. Für Zweiteres machen wir Gebrauch von Recurrent Neural Networks und Transformer-Architekturen.

Inhaltsverzeichnis

1	Konvertierung von Musik in Maschinenlesbare Daten	2
2	Komponistenklassifikation	3
2.1	Preprocessing	3
2.2	Klassifikation mit RandomForestClassifier	4
2.2.1	Erster Versuch	4
2.2.2	Modell zur Klassifizierung in die Epochen	5
2.2.3	Modelle zum Klassifizieren innerhalb der einzelnen Epochen	6
2.2.4	Verbinden der Modelle	6
2.3	Klassifizieren mittels Support Vector Classification (SVC)	7
3	Musikgenerierung	8
3.1	Musikgenerierung mit Recurrent Neural Networks	8
3.1.1	Preprocessing	8
3.1.2	Grundlegende Funktionweise	9
3.1.3	Ein erster Versuch	9
3.1.4	Zweiter Versuch mit $N = 1$	11
3.1.5	Eine neue Loss Funktion	12
3.2	Musikgenerierung mit Transformern	14
3.2.1	Preprocessing	14
3.2.2	Musikgenerierung mit Transformern	15
3.2.3	Die grundlegende Transformer-Architektur	15
3.2.4	Ein erster Versuch	16
3.2.5	Ein alter Bekannter	19

3.2.6	Ein Versuch von Data-Augmentation	20
3.2.7	Benchmarking	20

Quellen	26
---------	----

1 Konvertierung von Musik in Maschinenlesbare Daten

Bevor wir mit der Klassifikation und Generierung von Musikstücken beginnen können, müssen wir diese in ein Format umwandeln, welches von Maschinen gelesen und verarbeitet werden kann. Hierfür verwenden wir das MIDI-Format (Musical Instrument Digital Interface). MIDI-Dateien enthalten Informationen über Noten, Lautstärke, Tempo und andere musikalische Parameter, jedoch keine Audiodaten selbst. Siehe (Swift, 1997) für eine ausführliche Beschreibung des MIDI-Formats und seiner Struktur.

Im Rahmen dieses Projekts sind wir nur an den Notenwerten interessiert, sprich die Tonhöhe. Dabei wird jede Note durch eine (ganze) Zahl zwischen 0 und 127 dargestellt, wobei 60 dem mittleren C entspricht. Das bedeutet natürlich, dass wir keine Informationen über die Klangfarbe, das Instrument, die Lautstärke oder die Dauer (Rhythmus) der Noten haben. Insofern führen wir hier also eine Art Dimensionsreduktion durch und 'projizieren' die Musikstücke auf die Tonhöhe der Noten. Das würde dann in etwa so aussehen:



Abbildung 1: Musikstück nach Reduktion auf Tonhöhe der Noten

Man sieht hier noch eine weitere Diskrepanz: Die Noten sind nicht immer einzeln nacheinander gespielt, sondern teilweise auch gleichzeitig (Akkorde). Daher führen wir noch eine weitere Vereinfachung durch und ordnen die Noten in einen Vektor ein. Dabei geht natürlich wieder Information verloren. Wie genau das Preprocessing durchgeführt wird, ist für die beiden Aufgabenstellungen (Klassifikation und Generierung) unterschiedlich und wird in den jeweiligen Abschnitten beschrieben.

2 Komponistenklassifikation

2.1 Preprocessing

Für die Klassifikation verwenden wir das Dataset 'Classical Music MIDI as CSV', welches auf Kaggle zur Verfügung steht.

Das Dataset ist wie folgt aufgebaut: Es gibt 10 verschiedene Kategorien, in welche die Musikstücke eingeordnet werden. Gibt es zu einem Künstler mehr als 100 Stücke, so hat dieser seine eigene Kategorie. Von vielen Künstlern stehen aber weniger als 100 Werke zur Verfügung. Diese (sowie auch die Stücke der Komponisten mit mehr als 100 Werken) finden sich in der Kategorie 'ALL'. Es gibt eine Kategorie 'Varios - Título desconocido', was spanisch ist für 'Verschiedene - Titel unbekannt', wo sich Stücke befinden, bei welchen weder Titel noch Künstler bekannt ist.

In diesem Dataset gibt es folgende Kategorien:

1. ALL, mit allen Stücken aus diesem Dataset,
2. Bach, wobei hier 35.191 Datenpunkte vorhanden sind,
3. Beethoven, mit 29.124,
4. Chopin, mit 6.328,
5. Händel, mit 6.031,
6. Mozart, mit 25.430,
7. Scarlatti, mit 12.335,
8. Schubert, mit 16.685,
9. Varios - Título desconocido und
10. Vivaldi, mit 7.494.

Künstler mit weniger als 100 Stücken sind beispielsweise: Pachelbel, Jakobowski oder Strauss. Für weitere Informationen zum Aufbau des Datensets, siehe (Vincentloos, 2022).

Die einzelnen Werke kommen in Form von MIDI-Dateien, wobei dem Datensatz auch eine Funktion beigelegt ist, welche diese in CSV-Dateien umwandelt. Aus diesem extrahieren wir dann die Notenwerte für jedes Stück und geben diese in einen Vektor mit Werten in $[0, 127]_{\mathbb{Z}}$. Anschließend zerteilen wir die Vektoren in Abschnitte der Länge L , damit wir eine einheitliche Sequenzlänge über alle Komponisten haben. L repräsentiert natürlich einen Hyperparameter, welchen wir für diese Arbeit auf 100 gesetzt haben.

Da die Daten sehr unausgeglichene sind (so hat zum Beispiel Bach am meisten Stücke mit 35.191, Händel jedoch nur 6.031), muss das Dataset abschließend noch balanciert werden. Dabei gehen aber im schlimmsten Fall (von Bach) 29.160

Datenpunkte, also über 80% der Informationen verloren. Das ist natürlich nicht gut für die Klassifikation, wie sich später auch zeigen wird.

Im Allgemeinen haben wir in diesem Abschnitt die Kategorien 1 (ALL) und 8 (Título desconocido) ausgelassen, weil sie alle Stücke bzw. Stücke ohne Künstler beinhalten und wir das für die Klassifikation nicht verwenden können.

2.2 Klassifikation mit RandomForestClassifier

2.2.1 Erster Versuch

Zur Klassifikation verwenden wir einen Random Forest. Trainiert man diesen mit den Default-Werten (scikit-learn developers, 2025a), so erhält man eine Accuracy von 27%. Setzt man den Defaultwert von $n_estimators$ (die Anzahl der Bäume im Forest) höher, so verbessert sich die Accuracy. Bei beispielsweise 1.000 Bäumen hat man eine Genauigkeit von 31%, was besser aber immernoch schlecht ist. Das Problem, wenn man $n_estimators$ höher setzt, ist, dass es länger braucht zum Laden. Betrachtet man die Konfusionmatrix (2), kann man Folgendes erkennen:

- Stücke von zum Beispiel Schubert können gut klassifiziert werden.
- Zu großen Verwechslungen kommt es beispielsweise zwischen Händel und Bach.
- Im Allgemeinen sieht man aber, dass Musikstücke oft in der gleichen Zeitepoche, in welcher die Künstler gelebt haben, vertauscht werden.

Durch die Beobachtung, dass Werke, die der gleichen Epoche angehören öfter verwechselt werden, sind wir auf die Idee gekommen im zweiten Versuch über zwei (bzw. dann fünf) Modelle zu klassifizieren. Hier haben wir das Musikstück zuerst seiner Epoche und dann innerhalb dieser dem Komponisten zugeordnet.

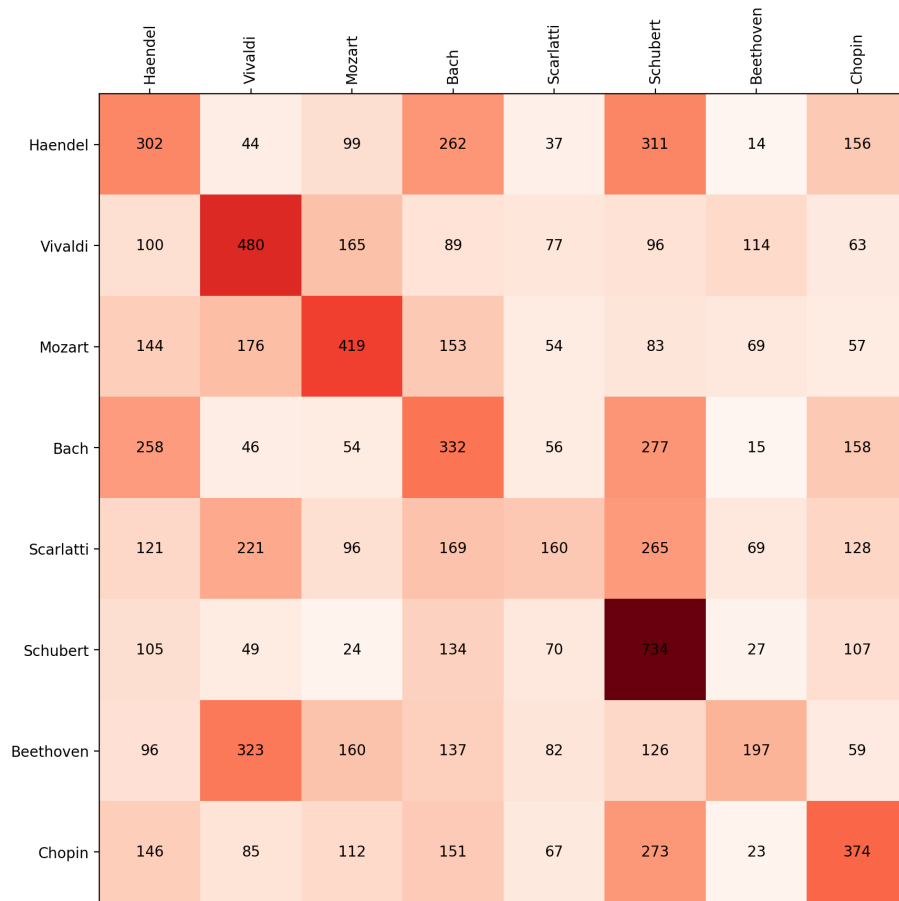


Abbildung 2: Klassifikation mit 1.000 Bäumen im Forest

2.2.2 Modell zur Klassifizierung in die Epochen

Wir trainieren zuerst nach den verschiedenen Epochen, in welchen die Künstler gelebt haben. Diese sind:

Barock : Scarlatti, Vivaldi, Bach, Händel, mit 61.051 Datenpunkten (Schmid, 2020),

Klassik : Mozart, Beethoven, mit 54.554 (Schmid, 2020) und

Romantik : Schubert, Chopin, mit 23.013 (Schmid, 2020).

Die Accuracy beim Einordnen der Stücke in ihre jeweiligen Epochen beträgt 55%. Auch hier verlieren wir wieder viele Daten beim Balancieren, da bei der Zuordnung zu den verschiedenen Zeitabschnitten große Unterschiede auftreten.

So hat die Romantik nur noch 1/3 der Daten, die beim Barock vorliegen. Dennoch ist das Problem zu Versuch 1, wo wir 80% verloren haben, besser geworden.

2.2.3 Modelle zum Klassifizieren innerhalb der einzelnen Epochen

Innerhalb der verschiedenen Epochen (Barock, Klassik und Romantik) werden die Werke ihren Komponisten zugeordnet. Das hat, wie man in der Grafik 3 erkennen kann, vor allem in der Klassik sowie Romantik gut funktioniert. Dort erreichen wir eine Accuracy von 67% bzw. 66%. Nur im Barock funktioniert es schlechter mit einer Genauigkeit von 43%, wobei vor allem Scarlatti und Vivaldi oft falsch eingeordnet werden.

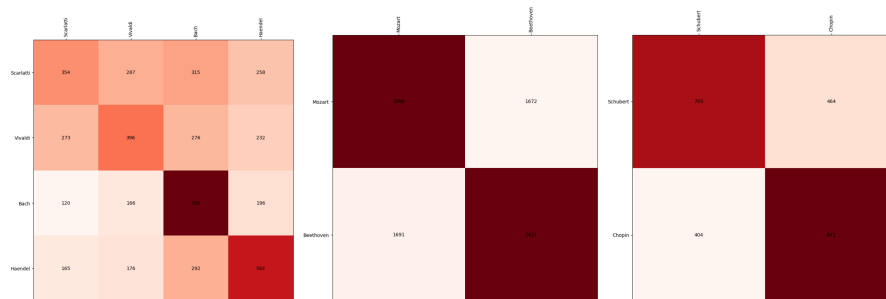


Abbildung 3: Links: Barock, Mitte: Klassik, Recht: Romantik

2.2.4 Verbinden der Modelle

Schlussendlich haben wir diese vier Modelle in folgendem Sinne verbunden:

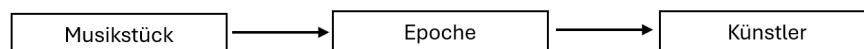


Abbildung 4: Verbinden der Modelle

Wie man aus dem Flowchart 4 entnehmen kann, wird ein Musikstück genommen und seiner Epoche zugeordnet. Innerhalb dieser Epoche wird es dann wieder seinem Künstler zugeteilt. Mit diesen fünf Modellen haben wir eine Accuracy von knapp 40% erreicht, was etwas besser ist, als zuvor. Dennoch ist sie nicht besonders gut, was daran liegt, dass schon beim Einteilen in die verschiedenen Epochen nur eine Genauigkeit von 55% erreicht wird. Somit wäre das dann auch die maximal mögliche Genauigkeit.

2.3 Klassifizieren mittels Support Vector Classification (SVC)

In einem neuen Versuch verwenden wir anstatt des RandomForestClassifiers eine Support Vektor Maschine, mit welcher wir dann Klassifizieren (gennant auch SVC für Support Vector Classification). Bei der SVC werden die Objekte in Klassen unterteilt, wobei diese durch Vektoren im Vektorraum dargestellt werden. Der Algorithmus versucht dann, die Objekte durch Hyperebenen so zu trennen, dass der Abstand der Vektoren, welche am nächsten an der Hyperebene liegen maximiert wird. Gibt man dem Modell später neue unbekannte Daten, versucht es, diese in die vorgelernten Klassen einzuteilen, anhand der Trennlinien, die zuvor antrainiert wurden (Lukas Exl, 2025).

Da dies aber vorallem für viele Daten sehr aufwändig ist, haben wir die Daten pro Künstler nur auf 603 (also 1/10 von dem, was wir aufgrund des Balancieren verwenden könnten) beschränkt. Ohne diese Maßnahme wäre es aufgrund der Ladedauer nicht möglich gewesen, einen RandomizedSearchCV anzuwenden, welcher die besten Parameter für die Support Vector Classification sucht.

Mit den Standardparametern der Support Vektor Klassifizierung (scikit-learn developers, 2025b) erhält man eine Accuracy von nur 18%.

Die Suche nach den besten Parametern ergibt eine Genauigkeit von 23%. Diese Parameter sind:

C : 0.5,

Kernel : Poly, (Diesen Parameter braucht man, da die SVC normalerweise nur linear sein würde. Das Modell versucht also die Daten linear zu trennen. Dies ist aber nicht möglich, da die gegebenen Daten nicht linear sind. Also verwendet man einen Kernel, um auch gekrümmte Schnittlinien zu erhalten.)

Degree : 3,

Gamma : auto, (Diesen Koeffizienten braucht man, wenn der Kernel 'rbf', 'poly' oder 'sigmoid' ist. Wir haben aber nur zwischen Poly, Sigmoid und Linear den besten Kerneotyp gesucht. Somit würde γ nur bei Poly oder Sigmoid verwendet werden.)

Tol : 0.01.

Dementsprechend ist die beste Methode für eine Klassifizierung, über die Zeitepochen die Stücke einzuteilen und dann innerhalb dieser nochmals spezifisch die Künstler zuzuordnen. Jedoch ist auch dieses nicht gut, da nur eine Genauigkeit von etwa 40% erreicht werden kann. Das Problem liegt dabei daran, dass sich die Stücke von der Abfolge der Noten (der Melodie) nicht weit genug unterscheiden, dass beim Trainieren ein Unterschied erkannt wird.

3 Musikgenerierung

3.1 Musikgenerierung mit Recurrent Neural Networks

3.1.1 Preprocessing

Für die Musikgenerierung verwenden wir das Bach-Chorales Dataset, welches ca. 400 Choräle von Johann Sebastian Bach enthält. Diese wurden aus MIDI-Dateien extrahiert und in ein geeignetes Format umgewandelt, welches sich an den vier Stimmen (Sopran, Alt, Tenor, Bass) orientiert. Jede Stimme wird dabei als eine Sequenz von Notenwerten (ganzen Zahlen) dargestellt, siehe das unten stehende Beispiel.

# note0	# note1	# note2	# note3
65	60	57	53
65	60	57	53
65	60	57	53
65	60	57	53
72	60	55	52
72	60	55	52
70	60	55	52
70	60	55	52

Abbildung 5: Repräsentation eines Chorals im Bach-Chorales Dataset

Wie bereits im ersten Abschnitt erwähnt, wollen wir die Notenwerte in einem Vektor darstellen, was wir in unserem Preprocessing dann auch tun. Somit wird jedes Stück nun als eine Abfolge von ganzen Zahlen in $[0, 127]_{\mathbb{Z}}$ dargestellt.

Nun stellt sich die Frage, wie wir diese Daten unserem RNN zuführen. Konkret müssen wir 3 Hyperparameter festlegen:

- N : Wie viele Noten sollen vorhergesagt werden?
- L : Wie viele vorherige Noten sollen betrachtet werden, um die nächsten Noten vorherzusagen?
- O : Das Offset, also wie viele Noten wir bei der nächsten Vorhersage überspringen.

Um das genauer zu erklären: Sei $S = [n_1, n_2, \dots, n_k]$ eine Sequenz von Notenwerten. Dann wäre unser erstes Input-Output-Paar für das Training:

$$[n_1, \dots, n_L] \rightarrow [n_{L+1}, \dots, n_{L+N}]$$

und unser zweites Paar:

$$[n_{1+O}, \dots, n_{L+O}] \rightarrow [n_{L+1+O}, \dots, n_{L+N+O}]$$

und so weiter.

3.1.2 Grundlegende Funktionweise

Bevor wir unsere verschiedenen Versuche vorstellen, wollen wir kurz erklären, wie die Musikgenerierung mit dem trainierten Modell funktioniert. Gegeben ein Anfangssequenz von Noten $[n_1, \dots, n_L]$, können wir das Modell verwenden, um die nächsten N Noten vorherzusagen. Dabei erweist es sich als nützlich, nicht deterministisch zu arbeiten, sondern den Output der softmax Schicht als Wahrscheinlichkeitsverteilung zu interpretieren und daraus eine Note zu sampeln, also NICHT immer die Note mit der höchsten Wahrscheinlichkeit zu wählen.

Diese vorhergesagten Noten werden dann an das Ende der Sequenz angehängt, und wir nehmen wieder die letzten L Noten, um die nächsten N Noten vorherzusagen. Diesen Prozess wiederholen wir, bis wir die gewünschte Länge des generierten Musikstücks erreicht haben.

Da es in Latex sehr schwierig ist, Audiodateien einzubetten, findet sich hier ein Link zu einem Kaggle Notebook, welches alle generierten Musikstücke mit Audiodateien enthält: <https://www.kaggle.com/code/kristiankelly/audiodateien-fuer-ausarbeitung>

3.1.3 Ein erster Versuch

Gerade stehen wir ganz am Anfang, wir können also nur raten, wie wir N, L, O wählen und welche Architektur wir verwenden sollen. Wir haben auf gut Glück $N = 31$ gesetzt (damit wir nicht immer auf einer Bass-Note enden). Für L scheint 4 eine gute Wahl zu sein, da wir ja 4 Stimmen haben. Für O haben wir uns für 1 entschieden, also immer eine Note weiter. Unsere Architektur ist ein einfaches RNN, welches wie strukturell für den Rest des Projekts verwendet werden, siehe Abbildung 6.

Wir heben folgende Dinge hervor:

- Wir verwenden eine Embedding-Schicht, um die Notenwerte in einen dichteren Vektorraum (in diesem Fall $\mathbb{R}^{64}$) abzubilden, was dem Netzwerk helfen soll, siehe z.B. Chollet, 2021 Kapitel 14. Es ist nämlich bei Noten wie bei Sprachen so, dass Noten, deren Abstände sehr kurz sind (z.B. C und C#) nicht unbedingt eine ähnliche Bedeutungen/Wirkung haben. Somit muss das Netzwerk diese Beziehungen selbst lernen.
- Wir verwenden zwei LSTM (Long Short-Term Memory) Zellen, welche sehr vereinfacht gesagt in der Lage sind, Langzeitabhängigkeiten in Sequenzen zu lernen, siehe z.B. Chollet, 2021 Kapitel 13 für eine ausführliche Beschreibung.
- Am Ende nehmen wir uns die letzten $N = 4$ Ausgaben und leiten diese über eine Dense-Schicht mit Softmax-Aktivierung, um eine Wahrscheinlichkeitsverteilung über die möglichen Notenwerte zu erhalten. Da die höchsten

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 31, 64)	5,632
lstm (LSTM)	(None, 31, 128)	98,816
dropout (Dropout)	(None, 31, 128)	0
lstm_1 (LSTM)	(None, 31, 128)	131,584
dropout_1 (Dropout)	(None, 31, 128)	0
lambda (Lambda)	(None, 4, 128)	0
dense (Dense)	(None, 4, 128)	16,512
dense_1 (Dense)	(None, 4, 88)	11,352

Total params: 263,896 (1.01 MB)

Trainable params: 263,896 (1.01 MB)

Non-trainable params: 0 (0.00 B)

Abbildung 6: Einfaches RNN mit $N = 31, L = 4, O = 1$

und tiefsten Notenwerte im Dataset nicht vorkommen, können wir auf 88 reduzieren, also die Tasten eines Klaviers.

Wir trainieren für 20 Epochen mit Early Stopping und erhalten eine val-accuracy von ca. 0.82 und eine test-accuracy von ca. 0.80. NICHT SCHLECHT! Ein generiertes Musikstück sieht in Abbildung 7 (die ersten zwei Takte sind vorgegeben, der Rest wurde generiert). Uns fallen zwei Dinge auf. Zum einen werden die Akkorde oft wiederholt, was jedoch zu erwarten war, da wir die Rythmus Komponente komplett ignorieren, weswegen eine ganze Note (= 4 Viertelnoten) als 4 gleiche Noten generiert wird. Zum anderen sehen (oder hören wir viel mehr), dass es dem Modell gelungen ist, gewisse Kadenzen (= typische Abfolgen von Akkorden) zu lernen, wie etwa die authentische Kadenz in Takte 6-7. Siehe Amon, 2005, Kapitel über Harmonie und die Funktionalität in Dur und Moll.

Trotzdem beinhaltet das generierte Stück viele unharmonische Übergänge, sprich Kadenzen, die Harmonietheoretisch 'keinen Sinn' ergeben. (Wieder verweisen wir bzgl. der genauen Bedeutungen dieser Aussage auf Amon, 2005). Ein Grund hierfür könnte sein, dass wir immer 4 Noten auf einmal vorhersagen und so das Modell vielleicht 'zu viel zu schnell' macht und daher Fehler auftauchen. Daher probieren wir im folgenden $N = 1$ aus.



Abbildung 7: Generiertes Musikstück mit $N = 32, L = 4, O = 1$

3.1.4 Zweiter Versuch mit $N = 1$

Wir behalten alles bei wie vorhin, nur dass wir jetzt $N = 1$ wählen, also immer nur eine Note vorhersagen. Wir trainieren für 30 Epochen mit Early Stopping und erhalten eine val-accuracy von ca. 0.85 und eine test-accuracy von ca. 0.83. BESSER! Ein generiertes Musikstück sieht man in Abbildung 8 (wieder sind die ersten zwei Takte vorgegeben, der Rest wurde generiert). Dieses Stück ist harmonisch deutlich besser gelungen (es klingt nichts 'falsch'). Jedoch ist dieses auch nur das beste aus 10 Versuchen, welches wir generiert haben. Die anderen hatten fast alle zumindest ein paar Stellen, die sehr dissonant klangen. Im Barock (die Epoche, in der Bach gelebt hat) waren solche unharmonischen Stellen eher unüblich und man hat sich an strikte harmonische Regeln gehalten, wie etwa den Kontrapunkt Tittel, 1959. Als nächsten Schritt wollen wir versuchen, diese Regeln irgendwie in das Modell zu integrieren, um konsistenter harmonische Stücke zu generieren.



Abbildung 8: Generiertes Musikstück mit $N = 32, L = 1, O = 1$

3.1.5 Eine neue Loss Funktion

Bis jetzt haben wir immer die Sparse-Categorical-Crossentropy als Loss Funktion verwendet. Nun wollen wir dissonante Fehler mehr bestrafen als konsonante. Dafür brauchen wir zuerst ein Grundwissen über Intervalle, die Grundbausteine der Harmonielehre. Ein Intervall ist der Abstand zwischen zwei Tönen, gemessen in Halbtonschritten. Man unterscheidet zwischen perfekten, konsonanten und dissonanten Intervallen Amon, 2005. Eine Übersicht über die wichtigsten Intervalle ist in der Tabelle 1 dargestellt. Ist also $v^{(n)} \in \mathbb{R}^{88}$ unsere Vorhersage für eine richtige Note $n \in [0, 88]_{\mathbb{Z}}$, wollen wir einen Vektor $s^{(n)} \in \mathbb{R}^{88}$ definieren, welcher die Einträge von v je nach Intervall zum vorhergesagten Ton bestraft. Wir definieren $s^{(n)}$ wie folgt:

$$s_i^{(n)} = \begin{cases} 0, & |i - n| = 0 \pmod{12} \text{ (Prime)} \\ w_1, & |i - n| = 5, 7 \pmod{12} \text{ (perfekte Konsonanz)} \\ w_2, & |i - n| = 3, 4, 8, 9 \pmod{12} \text{ (Konsonanz)} \\ w_3, & |i - n| = 1, 2, 6, 10, 11 \pmod{12} \text{ (Dissonanz)} \end{cases}$$

mit $0 < w_1 < w_2 < w_3$ positiven Gewichten. Unser neuer Loss für eine Vorhersage $v^{(n)}$ mit richtiger Note n ist dann:

$$\text{Loss}(v^{(n)}, n) = \text{SparseCategoricalCrossentropy}(v^{(n)}, n) + v^{(n)} \cdot s^{(n)}$$

Intervall	Halbtöne	Beispiel (von C)	Konsonanz
Prime	0	C - C	Perfekte Konsonanz
kleine Sekunde	1	C - C#	Dissonanz
große Sekunde	2	C - D	Dissonanz
kleine Terz	3	C - D#	Konsonanz
große Terz	4	C - E	Konsonanz
reine Quarte	5	C - F	Perfekte Konsonanz
verminderte Quinte / Tritonus	6	C - F#	Dissonanz
reine Quinte	7	C - G	Perfekte Konsonanz
kleine Sexte	8	C - G#	Konsonanz
große Sexte	9	C - A	Konsonanz
kleine Septime	10	C - A#	Dissonanz
große Septime	11	C - B	Dissonanz
Oktave	12	C - C	Perfekte Konsonanz

Tabelle 1: Musikalische Intervalle von Prime bis Oktave mit Konsonanz/Dissonanz Amon, 2005

wobei $\cdot$ das euklidische Skalarprodukt ist. Wir bemerken, dass der zweite Term genau dann 0 ist, wenn die Note korrekt vorhergesagt wurde, und umso größer wird, je 'dissonanter' die Vorhersage ist, also genau dass, was wir wollten.

Trainieren wir nun dasselbe Modell wie im vorherigen Abschnitt mit dieser neuen Loss Funktion, wobei wir $w_1 = 1, w_2 = 2, w_3 = 3$ setzen, erhalten wir nach 30 Epochen eine val-accuracy von ca. 0.85 und eine test-accuracy von ca. 0.84. Wir haben uns also dahingehend nicht wirklich verbessert. Aber wie schauen die generierten Musikstücke aus? Nunja, wir haben unser Ziel erfüllt: Die generierten Stücke sind harmonisch perfekt. Leider bedeutet 'perfekt' in diesem Fall, dass die ganze Zeit nur der erste Akkord (die Tonika) gespielt wird. Das gibt uns zwar das objektiv beste harmonische Stück, aber es ist natürlich unglaublich langweilig.

Wir starten einen zweiten Versuch mit $w_1 = 0.01, w_2 = 0.1, w_3 = 1$, also wir bewegen uns auf einer logarithmischen Skala. Die val-accuracy und die test-accuracy bleiben gleich, aber die generierten Stücke sind nun das, was wir uns erhofft haben: Harmonisch sinnvoll und abwechslungsreich (und das bei fast jeder Generierung). Ein Beispiel ist in Abbildung 9 dargestellt.



Abbildung 9: Generiertes Musikstück mit $N = 32, L = 1, O = 1$ und neuer Loss Funktion

3.2 Musikgenerierung mit Transformern

3.2.1 Preprocessing

Für die Transformer-Architektur passen wir das Preprocessing ein wenig an. Jeden Choral erweitern wir um einen SOS-Token (Start-Of-Sequence) am Anfang und einen EOS-Token (End-Of-Sequence) am Ende. Alle diese modifizierten Choräle konkatenieren wir dann zu einem einzigen Vektor D .

Sei $C_i = [n_{i,1}, \dots, n_{i,\ell}]$ der i -ten Choral, der Datenvektor D ergibt sich dann als

$$D = [\text{SOS}, \underbrace{n_{1,1}, \dots, n_{1,\ell_1}}_{C_1}, \text{EOS}, \text{SOS}, \underbrace{n_{2,1}, \dots, n_{2,\ell_2}}_{C_2}, \text{EOS}, \dots, \text{EOS}]$$

Wir schreiben $D = [t_1, \dots, t_n]$ (wobei $t_1 = \text{SOS}$, $t_2 = n_{1,1}$ usw.), die einzelnen t_s bezeichnen wir als Tokens.

Ein Input-Output-Paar (x, y) ist definiert als:

$$\begin{aligned} x &= [t_k, \dots, t_{k+\ell_{\text{SEQ}}-1}] \\ y &= [t_{k+1}, \dots, t_{k+\ell_{\text{SEQ}}}] \end{aligned}$$

Das Target y ist also die um eine Position verschobene Input-Sequenz.

Unter Berücksichtigung der drei Spezialtokens SOS, EOS und PAD erhalten wir eine Vokabulargröße V von 131.

3.2.2 Musikgenerierung mit Transformern

Der Generierungsprozess verläuft analog zu RNNs durch Sampling aus der Wahrscheinlichkeitsverteilung des Modells.

In unserem Fall gestalten wir die Generierung mit Transformern etwas flexibler: An die Anfangssequenz wird nur noch folgende Anforderung gestellt $[n_1, \dots, n_k]$, mit $0 \leq k \leq \ell_{\text{SEQ}}$ (wobei wir für $k = 0$ eine leere Liste meinen), das ist Konsequenz des SOS-Tokens; ℓ_{SEQ} ist dabei im Wesentlichen die maximale Sequenzlänge, die das Modell verarbeiten kann, wir wählen, falls nicht anders genannt, $\ell_{\text{SEQ}} = 512$.

Wie sich die Länge der Anfangssequenz auf die Vorhersage des Modells auswirkt, untersuchen wir in Unterunterabschnitt 3.2.7.

Ebenso lassen wir das Modell 'entscheiden', wann das generierte Stück endet; die Konsequenz des EOS-Tokens (um Endlosschleifen zu vermeiden, ist eine maximale Länge aber dennoch zu setzen).

Die Audiodateien finden sich unter folgenden Link: <https://www.kaggle.com/code/ferdibck/audiodateien-fuer-ausarbeitung-transformer>.

3.2.3 Die grundlegende Transformer-Architektur

Für die konkrete Struktur orientieren wir uns an Chollet, 2021, Da wir während der Inferenz nur Zugriff auf die bisherigen Noten haben, implementieren wir einen Decoder-Only-Transformer. Die in Abbildung 10 dargestellte Architektur lässt sich in folgende Komponenten unterteilen, wobei wir an entscheidenden Stellen Hyperparameter wählen müssen, die die Kapazität und das Lernverhalten des Modells beeinflussen:

- Wie im Ansatz mit RNNs werden die diskreten Input-Tokens in Vektoren der Dimension d_{emb} eingebettet. Um Informationen über die Positionen zu erhalten, addieren wir positionsabhängige Vektoren gleicher Dimension, in unserem Fall nutzen wir das in Vaswani et al., 2017 vorgestellte Positional-Encoding.
- Das Herzstück unseres Transformers ist natürlich die Multi-Head-Attention, die Anzahl der Attentions-Heads N_H können wir als weiteren Hyperparameter wählen. Entscheidend ist für uns vorallem die Maskierung: Da wir autoregressiv generieren, muss das Modell an Position t Informationen der noch zu generierenden Noten an $t + 1, \dots, \ell_{\text{SEQ}}$ erhalten.
- Nach der Attention-Schicht folgt ein einfaches Feed-Forward-Netzwerk, in unserem Fall bestehend aus zwei Schichten. Die Dimension innerhalb dieses Netzwerkes ist ein weiterer Hyperparameter d_{ffw} .
- Schließlich bildet die letzte Schicht auf einen Vektor der Dimension V ab, wir nutzen keine Aktivierungsfunktion, erhalten die sog. Logits als Output. Wendet man auf diese die Softmax-Funktion an, erhält man die Wahrscheinlichkeitsverteilung für das Noten-Sampling.

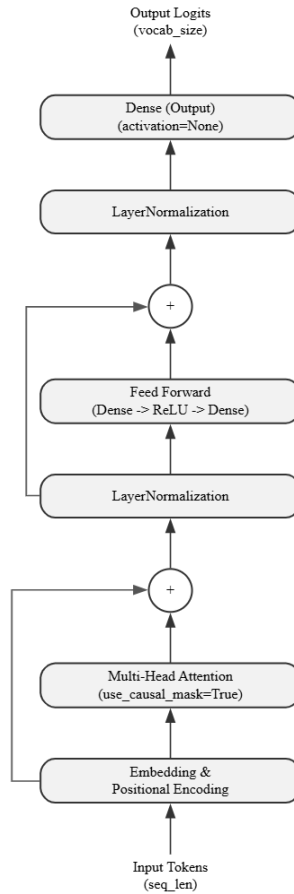


Abbildung 10: Grundlegende Struktur unseres Transformers

3.2.4 Ein erster Versuch

Nun stehen wir nicht mehr ganz am Anfang, wir orientieren uns an der Idee, die im Falle von RNNs erfolgreich war; unser Modell soll also **eine** Note aus $[0, 127]_{\mathbb{Z}}$ vorhersagen.

Gem. Unterunterabschnitt 3.2.3 haben wir 3 Hyperparameter zu bestimmen. Wir wählen

- $d_{\text{embed}} = 64$
- $N_H = 4$
- $d_{\text{ffw}} = 128$

und verwenden die Modellstruktur, welche in Abbildung 11 zu finden ist.

Layer (type)	Output Shape	Param #	Connected to
token_input (InputLayer)	(None, 512)	0	-
embedding (Embedding)	(None, 512, 64)	8,384	token_input[0][0]
add_6 (Add)	(None, 512, 64)	0	embedding[0][0]
attention (MultiHeadAttentio...	(None, 512, 64)	66,368	add_6[0][0], add_6[0][0]
add_7 (Add)	(None, 512, 64)	0	add_6[0][0], attention[0][0]
layer_normalizatio... (LayerNormalizatio...	(None, 512, 64)	128	add_7[0][0]
sequential_2 (Sequential)	(None, 512, 64)	16,576	layer_normalizat...
add_8 (Add)	(None, 512, 64)	0	layer_normalizat... sequential_2[0][...
layer_normalizatio... (LayerNormalizatio...	(None, 512, 64)	128	add_8[0][0]
logits (Dense)	(None, 512, 131)	8,515	layer_normalizat...

Total params: 100,099 (391.01 KB)

Trainable params: 100,099 (391.01 KB)

Non-trainable params: 0 (0.00 B)

Abbildung 11: Modell-Zusammenfassung mit $d_{\text{embed}} = 64$, $N_H = 4$ und $d_{\text{ffw}} = 128$

Wir trainieren ebenfalls für 20 Epochen mit EarlyStopping und ReduceLROnPlateau. Nach 15 Epochen wird das Training beendet und das beste Modell aus Epoche 10 übernommen: val-accuracy von ca. 84% und test-accuracy von ca. 82%. Wieder NICHT SCHLECHT. In Abbildung 12 findet sich das Mithilfe des gleichen Input für RNNs generierten Musikstückes; Abbildung 13 zeigt hingegen ein vollständig generiertes Stück, nur mit dem SOS-Token als Input.

Trotz ein paar harmonischen Ungereimheiten ist das erste Stück bis Takt 20 für das ungeschulte Ohr recht gelungen, ab Takt 21 sind die generierten Akkorde eher der Sorte 'Free Jazz'. Doch auch die Transformer-Architektur leidet unter Wiederholung der Akkorde.

Das vollkommen selbst generierte Stück ist hingegen deutlich unharmonischer, obwohl sich doch einige anhörliche Kadenzen finden lassen.



Abbildung 12: Generiertes Musikstück mit $\text{temp} = 1.2$, die ersten beiden Takte sind vorgegeben (A)



Abbildung 13: Generiertes Musikstück mit $\text{temp} = 1$, wir geben keine Noten vor (B)

Doch auch hier sei gewarnt: Die zwei generierten Stücke haben wir exemplarisch ausgewählt, es geht natürlich auch schlechter.

3.2.5 Ein alter Bekannter



Abbildung 14: Generiertes Musikstück mit $\text{temp} = 1.2$, die ersten beiden Takte sind wieder vorgegeben (C)



Abbildung 15: Generiertes Musikstück mit $\text{temp} = 1$, wir geben wieder keine Noten vor (D)

Wie oben gesehen hat der Transformer-Ansatz ebenfalls Probleme mit der Akkord-Harmonie, für RNNs hat hier ja die veränderte Loss-Funktion geholfen.

Wir trainieren also auch unseren Transformer (Abbildung 11) mit diesen, wobei wir $w_1 = 0.01$, $w_2 = 0.1$ und $w_3 = 1$ wählen.

Nach 20 Epochen erhalten wir das beste Modell aus Epoche 19 mit val-acc von etwa 83% und test-acc von etwa 82%, von den reinen Werten ist also keine Verbesserung zu erkennen. In der Generierung lässt sich sogar eher eine Verschlechterung erkennen, im Eye-Test scheint diese instabiler zu sein, die generierten Stücke finden sich in Abbildung 14 und Abbildung 15.

Aufgrund der häufig vorkommenden Dissonanzen trainieren wir das Modell erneut, dieses mal mit $w_1 = 0.01$, $w_2 = 1$ und $w_3 = 10$. Das Ergebnis ist noch ernüchternder, das Modell beendet jedliche Notensequenzen mit dem EOS-Token nahezu sofort.

3.2.6 Ein Versuch von Data-Augmentation

Unser Trainings-Korpus ist mit 220 000 Tokens lächerlich klein im Vergleich zur heutigen Menge von Trainingsdaten für Transformer-Modelle. Vielleicht helfen auch in unserem Fall mehr Daten.

Das Augmenting funktioniert ganz einfach, für jeden Choral $C_i = [n_{i,1}, \dots, n_{i,\ell}]$, fügen wir auch $C_i + t = [n_{i,1} + t, \dots, n_{i,\ell} + t]$ in unseren Datenvektor D hinzu; wir verschieben die Noten einfach um t Halbtonschritte nach oben oder unten. Wir nutzen $t = -5, -4, -3, \dots, 6$.

Als weiteren Wermutstropfen erhalten wir auch mit Data Augmentation nur eine val-acc bei 84% und eine test-acc bei 83%. Die Generierung läuft allerdings wieder etwas stabiler als in Unterunterabschnitt 3.2.5 (Zur Info: aufgrund der Trainingsdauer haben wir für dieses Modell $\ell_{\text{SEQ}} = 256$ gewählt).

3.2.7 Benchmarking

Abschließend wollen wir mit dem ersten Modell noch ein paar 'Benchmarks' auf dem Test-Set durchführen.

Zunächst betrachten wir für eine fixe Input-Länge die Generierung als Klassifikationsproblem, Abbildung 16 zeigt die zugehörige Confusion-Matrix.

Uns fallen zwei Dinge auf: Zunächst werden wenige Tokens tatsächlich verwendet, das ist allerdings nicht verwunderlich; sehr tiefe bzw. hohe Tonhöhen kommen in Musik (jeglicher Zeit) kaum vor. Diesen 'Beschränkung' hat auch unser Modell gelernt, ebenfalls dominiert die Hauptdiagonale die Vorhersagen deutlich, wir sagen also oft die korrekte Note vorher. Betrachtet man die Matrix genauer fallen die zwei symmetrischen Nebendiagonalen auf (die untere beginnt bei etwa (36, 48) und endet bei etwa (66, 78)), die Nebendiagonalen liegen also interessanterweise ± 12 Token-IDs unter bzw. über der Hauptdiagonale. Das sind genau 12 Halbtonschritte, also eine Oktave! Die Vorhersage ist also in diesem Fall harmonisch korrekt, wenn auch im falschen Intervall.

Abbildung 17 zeigt ein nahezu identisches Bild. Zum einem bestätigt das die Robustheit des Modells, doch ergibt sich daraus auch eine ernüchternde Konsequenz, Beziehungen von Noten über lange Zeiten sind nicht vorhanden oder

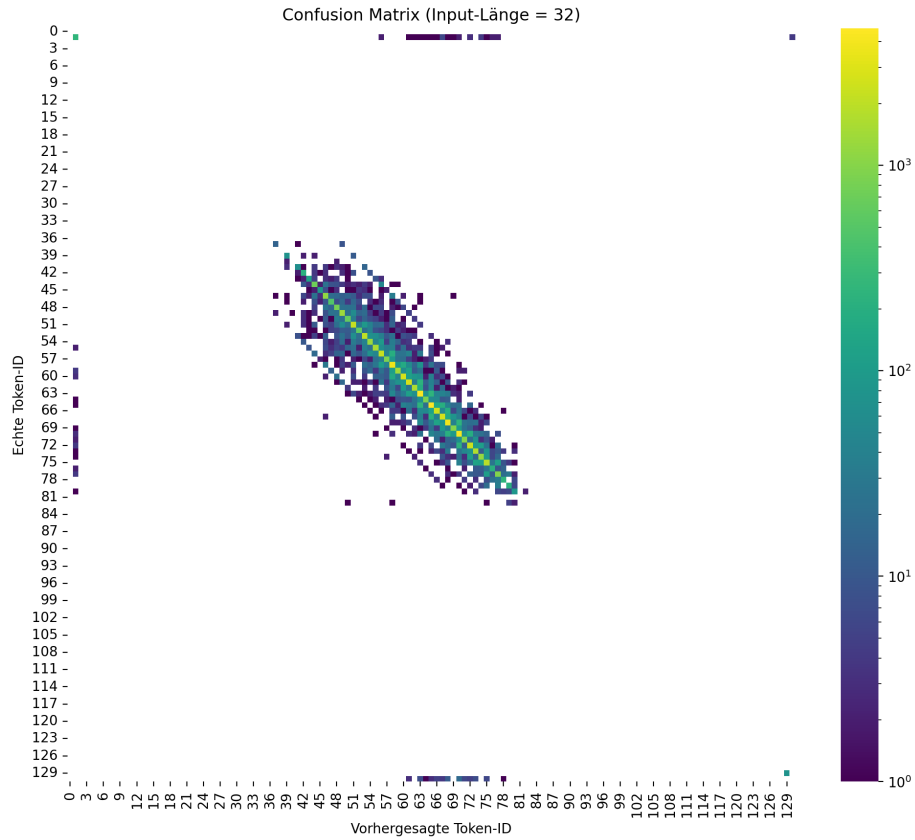


Abbildung 16: Confusion-Matrix, Input-Länge von 32

werden vom Modell nicht gelernt; doch gerade darin haben Transformer-Modell ihren Vorteil gegenüber anderer Methoden, siehe Vaswani et al., 2017. Das ist vermutlich auch der Grund für die nahezu identische Leistung gegenüber RNNs. Abbildung 18a, Abbildung 18b, Abbildung 19a und Abbildung 19b zeigt die Entwicklung der Confusion-Matrizen mit länger werdender Input-Länge von 1, 2, 3, 4. Zusammen mit Abbildung 20 bestätigen diese die Beobachtungen; ab einer Input-Länge von 4 ist die Vorhersage des Modells praktisch fix, die Confusion-Matrix ist nahezu identisch zu den Matrizen mit Input-Länge von 32 und Input-Länge von 128. Das Modell profitiert also kaum von den Vorteilen der Transformerarchitektur.

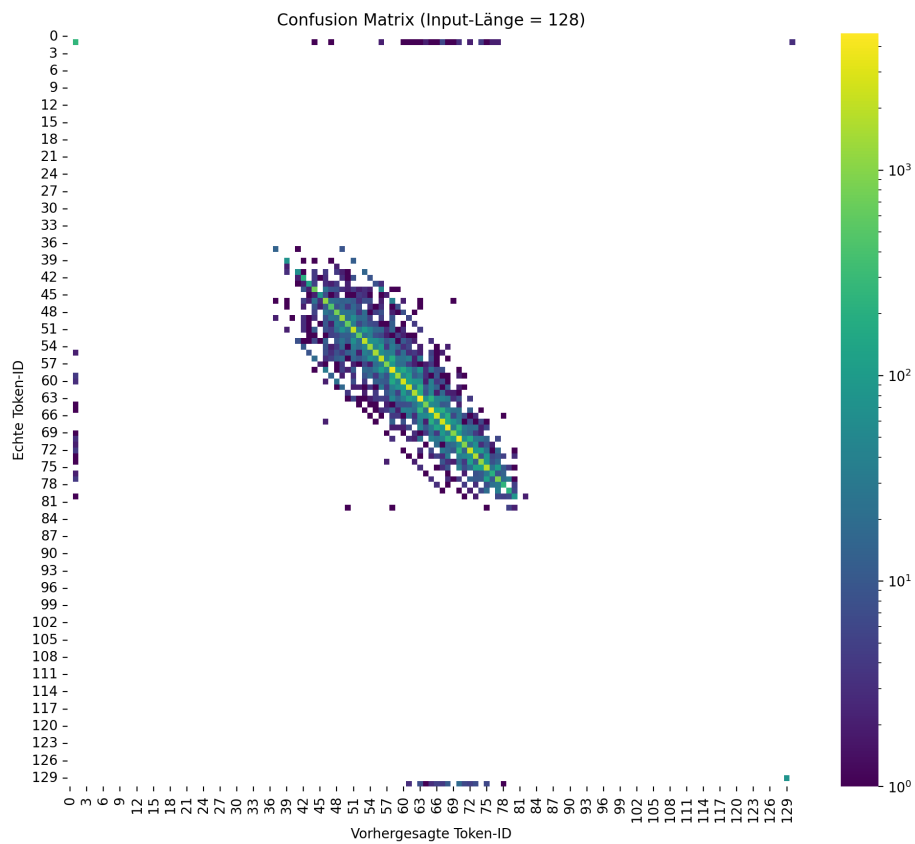
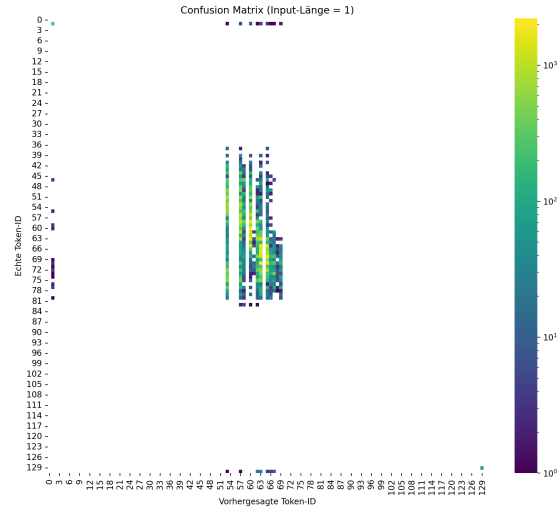
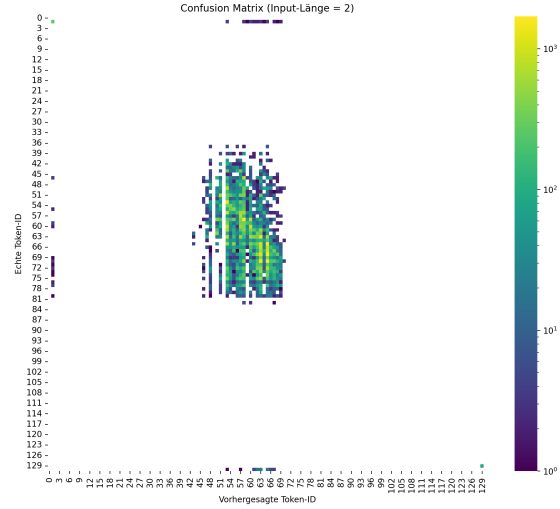


Abbildung 17: Confusion-Matrix, Input-Länge von 128

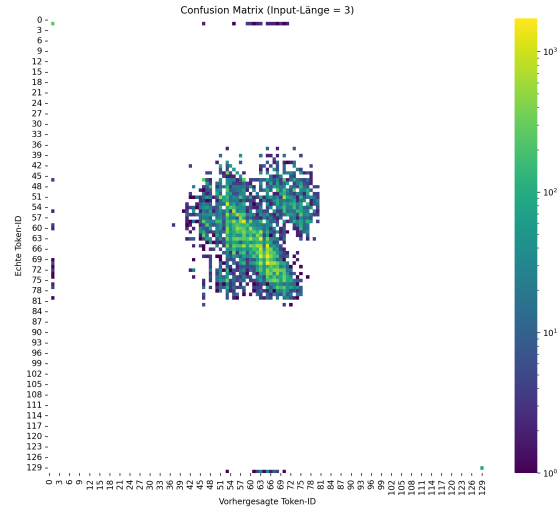


(a) Input-Länge von 1

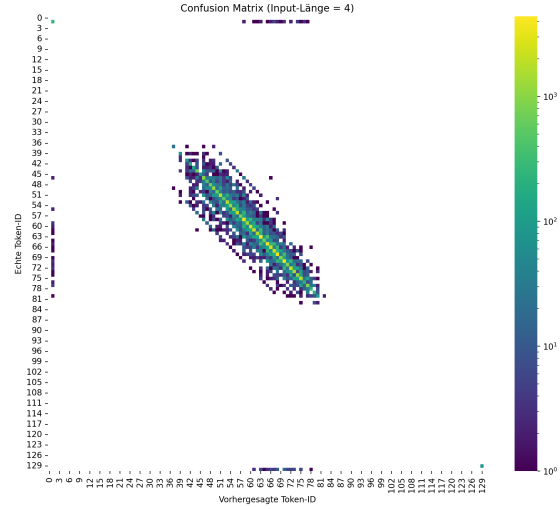


(b) Input-Länge von 2

Abbildung 18: Entwicklung der Confusion-Matrizen (Teil 1)



(a) Input-Länge von 3



(b) Input-Länge von 4

Abbildung 19: Entwicklung der Confusion-Matrizen (Teil 2)

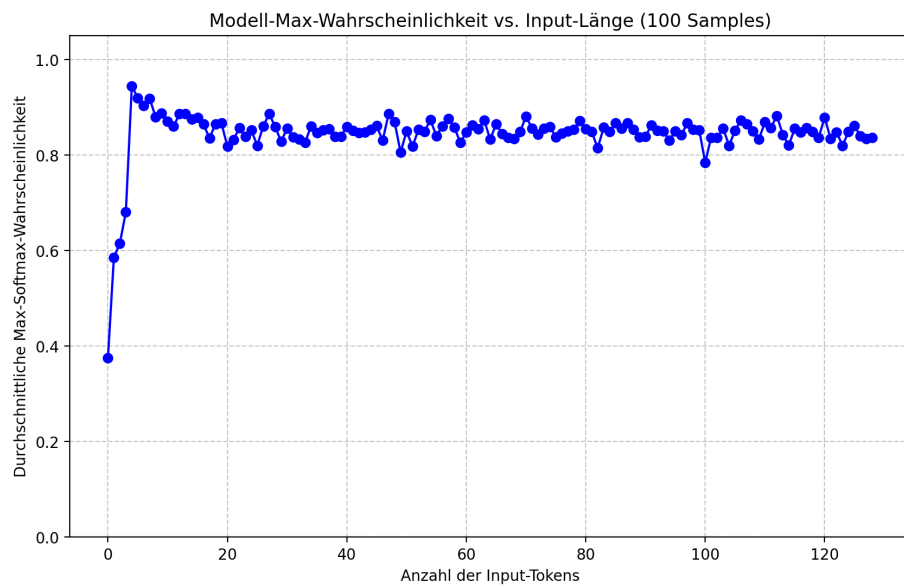


Abbildung 20: Maximale Wahrscheinlichkeiten für den nächsten Token in Abhängigkeit der Input-Länge

Quellen

- Amon, R. (2005). *Lexikon der Harmonielehre: Nachschlagewerk zur durmolltonalen Harmonik mit Analysechiffren für Funktionen, Stufen und Jazz-Akkorde*. Springer.
- Chollet, F. (2021). *Deep Learning with Python* (Second). Manning. https://www.manning.com/books/deep-learning-with-python-second-edition?a_aid=keras
- Lukas Exl, S. S. (2025). Angewandtes Maschinelles Lernen. *ML Skriptum*.
- Schmid, W. (2020). *Epochen der Musikgeschichte*. Helbling Verlag.
- scikit-learn developers. (2025a). Random Forest [Accessed: 2026-02-20]. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>
- scikit-learn developers. (2025b). Support Vector Classification [Accessed: 2026-02-20]. <https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html>
- Swift, A. (1997). Standard MIDI File Format [Accessed: 2026-01-07]. <https://web.archive.org/web/20120830211425/http://www.doc.ic.ac.uk/~nd/surprise.97/journal/vol1/aps2/>
- Tittel, E. (1959). *Der neue Gradus: Textteil* (Bd. 1). Doblinger.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, 30.
- Vincentloos. (2022). Classical Music MIDI as CSV [Accessed: 2026-02-14]. <https://www.kaggle.com/datasets/vincentloos/classical-music-midi-as-csv?select=README.txt>